Your Project Dashboard

Nicolas Cachanosky · Department of Economics and Finance · ECON-5371

Why start with deployment

5 pts

pass/fail: deployed and meets the minimum, or zero

A beautiful dashboard that only runs on your laptop scores zero is not very useful.

1. Get a skeleton running on **your** data
2. Put it **online**
3. Fill in the model results over the next weeks. Every push updates the live site

The reference repo

You have a sample Quarto dashboard on `github.com/ncachanosky/ECON-5371-lab-dashboard`
A two-page dashboard built on the Lab 4 VAR system. It's live, so open it in a browser tab now.

The minimum content checklist

To pass, your dashboard needs **all six**:

#	Requirement
1	**Deployed:** a live `github.io` URL, linked from your repo README
2	**Value boxes:** observations, sample period, and at least one key model number. All computed, none typed
3	**Raw series plot:** labeled, with units
4	**Main model results:** estimates or test results in a table
5	**Key result figure:** your forecast (fan chart) or main result (IRFs, volatility, break dates…)
6	**Links** to your repository and your paper

01

Anatomy of a dashboard

The reference dashboard, taken apart in three slides

Four pieces, every time

1

Data

What you're showing

2

Computation

What you did to it: a model, a test, a forecast

3

Layout

How it's arranged on the page

4

Interactivity

Optional: letting the viewer choose what they see

Why Quarto

Quarto dashboards are **static HTML**: no server to run, no new framework. `format: dashboard` in the YAML is the only new key; the rest is the `.qmd` authoring you've done all semester. They deploy to GitHub Pages exactly like your Week 4 website.

The layout grammar

Headings, not code, define the layout:

Markup	Creates
`# Overview`	A **page**, linked from the navbar
`## Row {height=55%}`	A **row** on that page
`### Column {width=40%}`	A **column** inside a row
`## Row {.tabset}`	A row whose cards become **tabs**
`#| title: "..."`	A titled **card** around one cell's output
`#| content: valuebox`	A compact **stat card**

A value box

```
```{python}
#| content: valuebox
#| title: "Observations"
#| icon: calendar3
#| color: primary
dict(value=len(df))
```
```

The cell returns a dictionary; Quarto turns `value` into a big number on a colored card.

Icons come from Bootstrap Icons (`icons.getbootstrap.com`).

`primary` and `secondary` map to UTEP Blue and Silver through the theme file; use `"#FF8200"` for Orange.



What it renders as

Three rules from building the reference

1. Heights on a page must sum to 100%

Quarto doesn't raise an error when they don't. It squeezes everything to fit, and the dashboard looks cramped. Re-add the heights every time you add a row.

2. Value boxes are for short content

A long string wraps and gets clipped. Shorten the content (`'85–'24`, not `1985-01-01 to 2024-10-01`) instead of fighting the box.

3. In a tabset, every cell is its own tab

To put several cells in one tab (a dropdown and the image it controls), group them under a `###` heading.

02

From reference to your project

Copy the skeleton, keep the rule, show what your question needs

Where the dashboard lives

Inside your **project repository**, in its own folder, next to everything else the repository rubric asks for.

Two new folders

`dashboard/` is what you write. `docs/` is what `quarto render` writes and what GitHub Pages serves.

```
your-project-repo/
├── data/
│   ├── raw/
│   └── clean/
├── code/
├── output/
├── paper/
├── dashboard/          ← new today
│   ├── _quarto.yml
│   ├── index.qmd
│   └── dashboard-theme.scss
├── docs/               ← created by render
└── README.md
```

Copy three files

From the reference repo's `dashboard/` folder into yours:

- `_quarto.yml`: sends the rendered site to `../docs`
- `index.qmd`: the dashboard itself
- `dashboard-theme.scss`: UTEP colors and fonts

Then point the data path at **your** cleaned data:

```
# Quarto runs index.qmd from inside
# dashboard/, so data is one level up
DATA_PATH = Path(
    "../data/clean/your_series.csv")
```

Forward slashes, always

`"../data/clean/file.csv"` works on Windows, Mac, and Linux. A backslash path (`"..\data\clean"`) does not: Python reads backslashes as escape characters.

The rule that matters most

Fit once, at the top. Compute every number you display.

The reference has a hidden setup cell before the first page:

````
```{python}
#| include: false
df = pd.read_csv(DATA_PATH, parse_dates=["date"]).set_index("date")

model = VAR(df[VARIABLES], freq="QS")
lag_order = int(model.select_order(MAX_LAGS).aic)
fit = model.fit(lag_order)
max_eig_modulus = 1 / np.abs(fit.roots).min()
```
````

`include: false` runs the code but shows nothing. Every card below it reads from `fit`, on any page.

What to show: by project type

Project	Value boxes	Key figure	Results table
Forecasting (ARIMA/ETS)	N, sample, chosen order	Fan chart, test period	RMSE/MAE vs. benchmark; DM
Structural breaks	N, sample, break date(s)	Series + break lines	Chow / QLR / Bai-Perron
VAR / SVAR	N, lag order, stability	IRFs by shock	Granger causality, FEVD
GARCH	N, persistence $\alpha + \beta$	Returns and conditional volatility	Estimates with SEs
State space / regimes	N, sample	Filtered state / regime probs	Estimates

03

Hands-on: page 1 on your data

Ten minutes

Ten minutes: get the Overview page working

1. Copy the three files into `dashboard/`
2. Set `DATA_PATH`; replace `VARIABLES` with your series names
3. **Comment out** the model code in the setup cell for now. Keep only the data loading
4. Delete the `# Model Results` page and the two model value boxes on page 1. They come back when your model does
5. Keep: three value boxes (observations, date range, variables) and the raw series plot
6. Preview it

```
cd dashboard
quarto preview
```

Leave it running

`preview` opens a browser and re-renders every time you save.

Check the plot before moving on

````
```{python}
#| title: "Your Series Title (units)"
fig, ax = plt.subplots(figsize=(11, 4))
for var in VARIABLES:
    ax.plot(df.index, df[var], label=var,
            linewidth=1.2)
ax.set_ylabel("Units here")
ax.legend(loc="upper left", frameon=False)
ax.set_xlim(df.index.min(), df.index.max())
plt.show()
```
````

- The card **title** names the series; the y-axis label gives units
- `set_xlim` to the data range, the same rule as in the textbook figures
- Plot looks squashed? Check rule 1: do the row heights still sum to 100% after you deleted the model value boxes?

04

Deploy

Same route as your Week 4 website

One setting sends output to docs/

```
# dashboard/_quarto.yml
project:
  type: default
  output-dir: ../docs
```

GitHub Pages can serve a site from a `/docs` folder on `main`. This is the same route as your Week 4 website.

The only difference is the `../`: the dashboard source lives in `dashboard/`, but the rendered site goes to `docs/` at the repo root.

render publishes, preview doesn't

```
cd dashboard
quarto render
```

Writes `docs/index.html`, the file GitHub Pages serves.

```
cd dashboard
quarto preview
```

Local editing only. **Never touches `docs/`.** If the live site isn't changing, you probably only previewed.

Create the `.nojekyll` file once, so GitHub serves the files as they are:

```
touch ../docs/.nojekyll
```

Push, then turn on Pages

```
cd ..
git add .
git commit -m "Add project dashboard"
git push
```

One-time setting on GitHub:

1. Your project repo → **Settings → Pages**
2. Source: **Deploy from a branch**
3. Branch: **main**, folder: **/docs**, then Save

One to two minutes later: `https://<username>.github.io/<repo-name>/`

Public repo required

Free GitHub Pages only serves public repositories, the same constraint as Week 4.

The gotcha that only appears online

The impulse-response dropdown shows images that Python **creates** during the render. Locally, everything works. Render into `docs/` without one extra line, and every image is missing from the live site.

Why

When rendering into `docs/`, Quarto copies only the files it can **see** the page using. It can't see a file your code writes, or a filename that JavaScript builds on the fly (`images/irf_${shock}.png`).

The fix, one YAML key in `index.qmd`:

```
resources:
  - images/
```

Check: after `quarto render`, does `docs/` contain everything the page shows? If a file isn't in `docs/`, it doesn't exist online.

When it doesn't work

Symptom	Likely cause
404 at the `github.io` URL	Pages not enabled, wrong branch/folder, or repo is private
Live site shows an old version	You ran `preview`, not `render`, or didn't push `docs/`
Site loads but looks unstyled	Missing `docs/.nojekyll`
Images broken online, fine locally	File not in `docs/`: add it to `resources:`
`FileNotFoundError` on render	Data path not relative to `dashboard/`; should start with `../`
Cards cramped or overlapping	Row/column heights don't sum to 100%
`ModuleNotFoundError` on render	Quarto found the wrong Python: set `QUARTO_PYTHON` to your `econ5371` env

05

Wrap-up

Link it, both ways

Add your repo and paper links to the dashboard navbar:

```
format:
  dashboard:
    nav-buttons:
      - icon: github
        href: https://github.com/<username>/<repo-name>
      - icon: file-earmark-text
        href: https://github.com/<username>/<repo-name>/blob/main/paper/paper.pdf
```

And the live URL to the top of your `README.md`:

```
**Dashboard:** https://<username>.github.io/<repo-name>/
```

Checklist: where you should be today

#	Requirement	Today
1	Deployed, URL in README	✓
2	Value boxes, all computed	Partial: data boxes now, model box later
3	Raw series plot with units	✓
4	Main model results table	As your model is finalized
5	Key result figure	As your model is finalized
6	Links to repo and paper	✓

From here on

Every time your model changes: edit the setup cell, `quarto render`, push. The live dashboard follows your project automatically.